STM32H750 电赛智能车 – 原理与实现

3.4: 视觉模块AI模型部署

InnoLegend 技术团队

目 录

CONTENTS

1 本讲概述

2 数据获取与模型训练

3 模型部署与使用

4 实战环节

第一部分

本讲概述

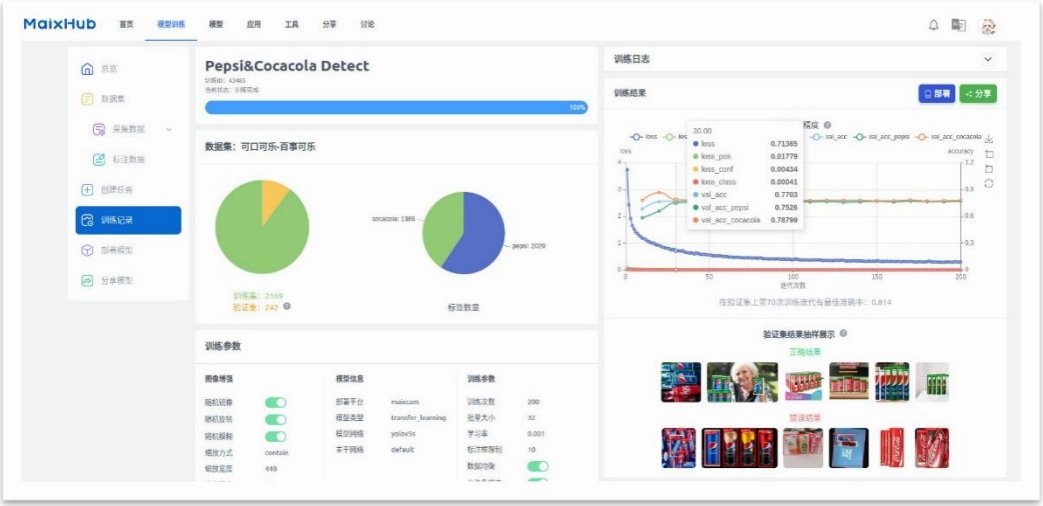
Introduction to this lecture

1.1 内容概述

InnoLegend

你将会学到的：

- AI模型使用过程中的注意事项。
- 实践时踩过的坑。
- AI模型从零开始训练到部署的过程。
- AI模型的使用方法。



课程不会讲述的：

- 过多机器学习相关的理论知识

1.2 原理概述

InnoLegend

◘模型是什么？

一组数据结构，保存了一个网络的形状，以及里面的参数。 通常，这个模型的数据被保存成不同的形式，比如.h5 .tflite .kmodel等文件。使用不同的软硬件去解释后使用。

◘ 运行模型

OpenMV依靠CPU运行模型，且模型必须先量化成INT8.tflite，性能较差。

K210提供 KPU (Kendryte Proccess Unit)，加速计算，集成了推导模型的代码，性能较Openmv更强，简单易用。

第二部分

K210模型训练与部署

K210 Model Training and Deployment

2.1 K210图像标注工具

InnoLegend

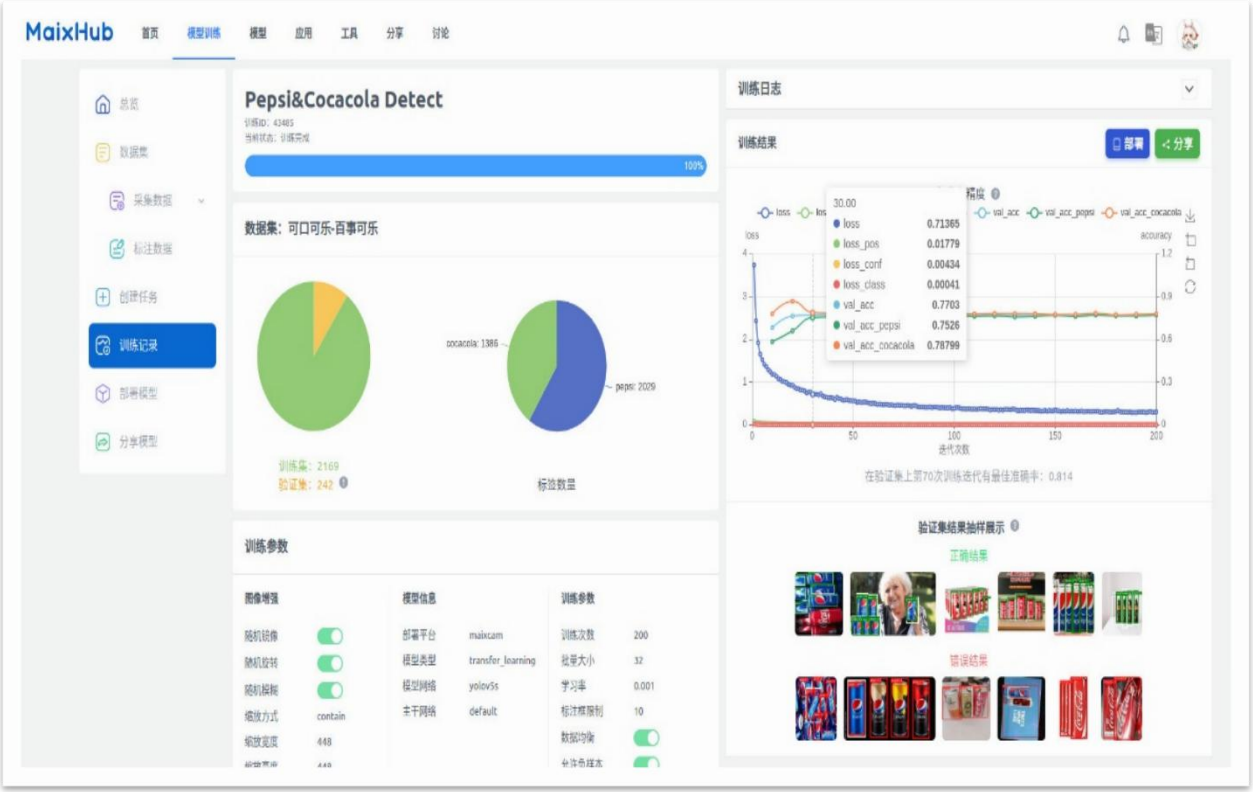
- 推荐使用方法：

在线训练网址：https://maixhub.com/
简单快捷，支持一键导出，全部免费。

- 其他可选项：

开源软件：labelme
标注功能更加全面，但在当前的模型中不可用。

`pip install labelme`



2.2 数据准备的注意事项

InnoLegend

```
img_cnt = 0
def shot():
    global img_cnt
    sensor.skip_frames(time = 2000)
    img_name = "example" + str(img_cnt) + ".jpg“
    sensor.snapshot().save("/sd/number/" + img_name)
    lcd.display(image.Image("/sd/number/" + img_name))
    img_cnt += 1
```

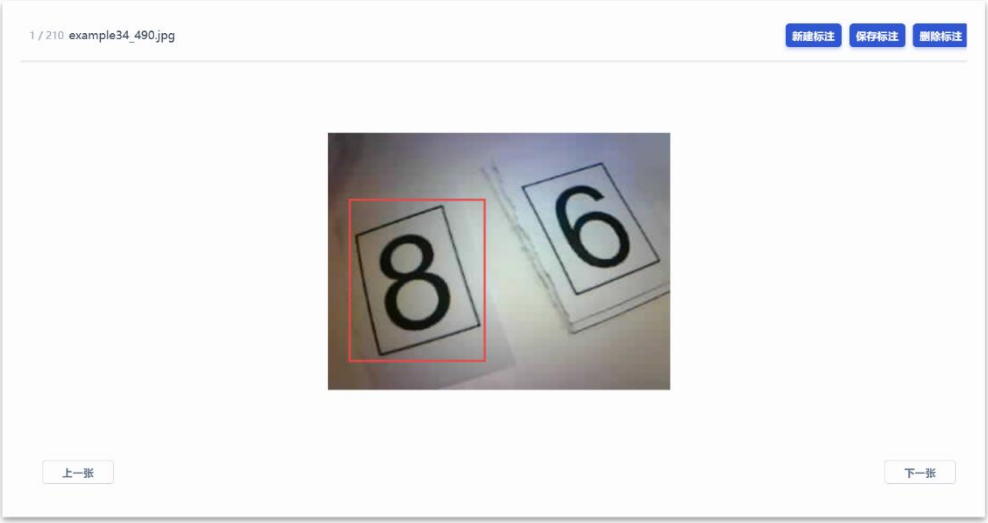
注意!

- 由于硬件情况的不同，轻易使用其他的模型或者数据集可能导致识别正确率下降。
- maixhub网站上的模型输入的图片大小为224 * 224的。直接调整图片大小会出现报错，需要使用其他的方法处理。
- maixhub的最大数据集不允许超过20MB, 不允许超过3k张照片。

2.3 图像标注

InnoLegend

- **习惯上，数**据集会被划分为80%作为训练集，20%作为验证集。
- **训练集**：模型通过训练集进行训练，学习数据中的模式或规律。
- **验证集**：验证集用于在训练过程中对模型进行评估，避免模型对训练集的过拟合。通过验证集，可以帮助调整模型的超参数，并确定何时停止训练。



2.4 K210模型训练

InnoLegend

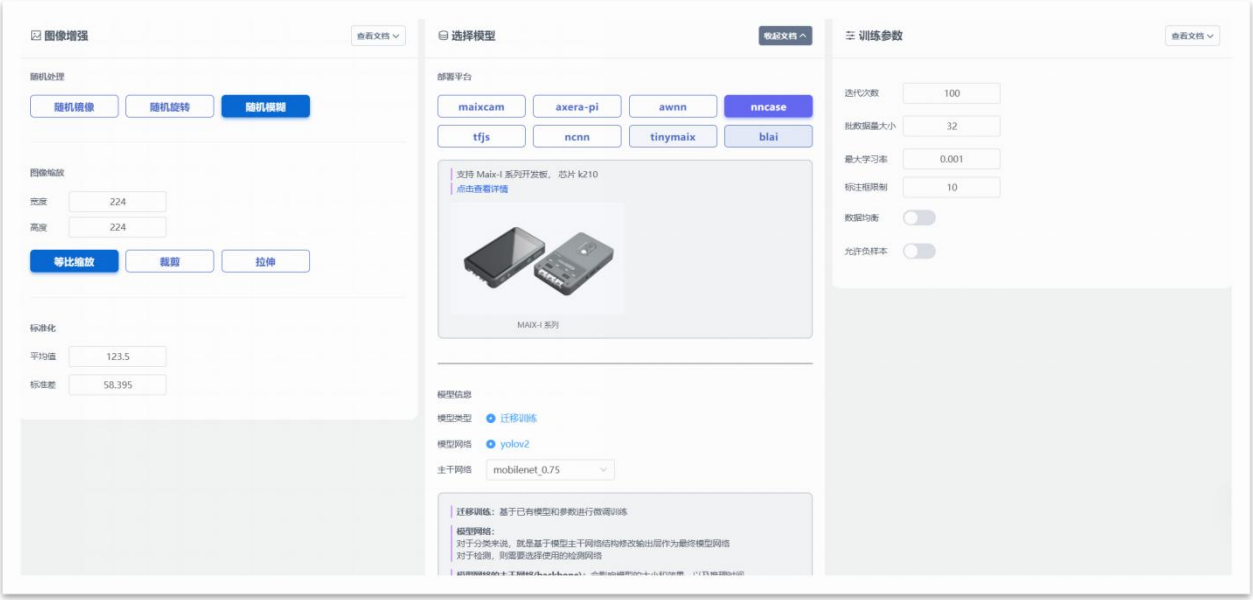
- 推荐使用方法：

maixHub在线训练。

缺点：

高峰期训练排队

模型输入大小难以调整。



- 其他可选项：

本地训练： https://wiki.sipeed.com/soft/maixpy/zh/course/ai/train/local.html

环境配置繁琐！不推荐!

2.5 K210模型导出与导入

InnoLegend

- 导出模型后，会得到三个文件：

  main.py —— 运行的示例程序，包含必要的参数

  XXXX.kmodel —— 神经网络模型

  report.json —— 模型参数

```
input_size = (224, 224)
labels = ['8', '6', '2', '4', '1', '3', '5', '7']
anchors = [0.78, 1.09, 2.45, 3.31, 2.03, 2.47, 1.75, 1.75, 1.09, 1.44]
```

- 使用模型时，一般将模型放在两种位置：

  –“/sd/m.kmodel”

  –flash中（从0x300000处开始）

```
# main(anchors = anchors, labels=labels, model_addr=0x300000, lcd_rotation=0)
main(anchors = anchors, labels=labels, model_addr="/sd/model-180664.kmodel")
```

2.6 kmodel模型的使用

InnoLegend

```
task = kpu.load(model_addr)  # 载入模型
kpu.init_yolo2(task, 0.5, 0.3, 5, anchors) # threshold:[0,1], nms_value: [0, 1]
while(True):
    img = sensor.snapshot()
    objects = kpu.run_yolo2(task, img)
    if objects:
        for obj in objects:
            pos = obj.rect()
            img.draw_rectangle(pos)
    lcd.display(img)
```

- KPU.init_yolo2(kpu_net, threshold, nms_value, anchor_num, anchor)

  - kpu_net: kpu网络对象, 即加载的模型对象, KPU.load()的返回值
  - threshold: 概率阈值， 只有是这个物体的概率大于这个值才会输出结果， 取值范围：[0, 1]
  - nms_value: 为了防止同一个物体被框出多个框，当在同一个物体上框出了两个框，这两个框的交叉区域占两个框总占用面积的比例 如果小于这个值时， 就取其中概率最大的一个框
  - anchor_num: anchor 的锚点数， 这里固定为 len(anchors)//2
  - anchor: 锚点参数与模型参数一致。

2.6 kmodel模型的使用

InnoLegend

```
task = kpu.load(model_addr)  # 载入模型
kpu.init_yolo2(task, 0.5, 0.3, 5, anchors) # threshold:[0,1], nms_value: [0, 1]
while(True):
    img = sensor.snapshot()
    objects = kpu.run_yolo2(task, img)
    if objects:
        for obj in objects:
            pos = obj.rect()
            img.draw_rectangle(pos)
    lcd.display(img)
```

- obj对象
  - obj.reac() : 返回pos，为一个四个元素的数组，分别代表预测的目标的左上，右下像素位置。
  - obj.classid() : 返回物体编号。
  - obj.value(): 返回物体的置信度。

```
img.draw_string(pos[0], pos[1], "%s : %.2f" %(labels[obj.classid()], obj.value()), scale=2,
color=(255, 0, 0))
```

2.7 注意事项

InnoLegend

- 正常使用的时候会数据类型转换错误

  原因：内存不足!

- 调试时在模型处会出现内存不足

  原因：没有释放模型，内存泄漏!

- 网站使用的模型大小为1.8MB，即使使用最大的固件依然可以正常使用。

```
while True:
    try:
        pass
        gc.collect()
    except Exception as e:
        kpu.deinit(task)
        raise
```

第二部分

OpenMV模型训练与部署

OpenMV Model Training and Deployment

3.1 OpenMV数据获取与标注

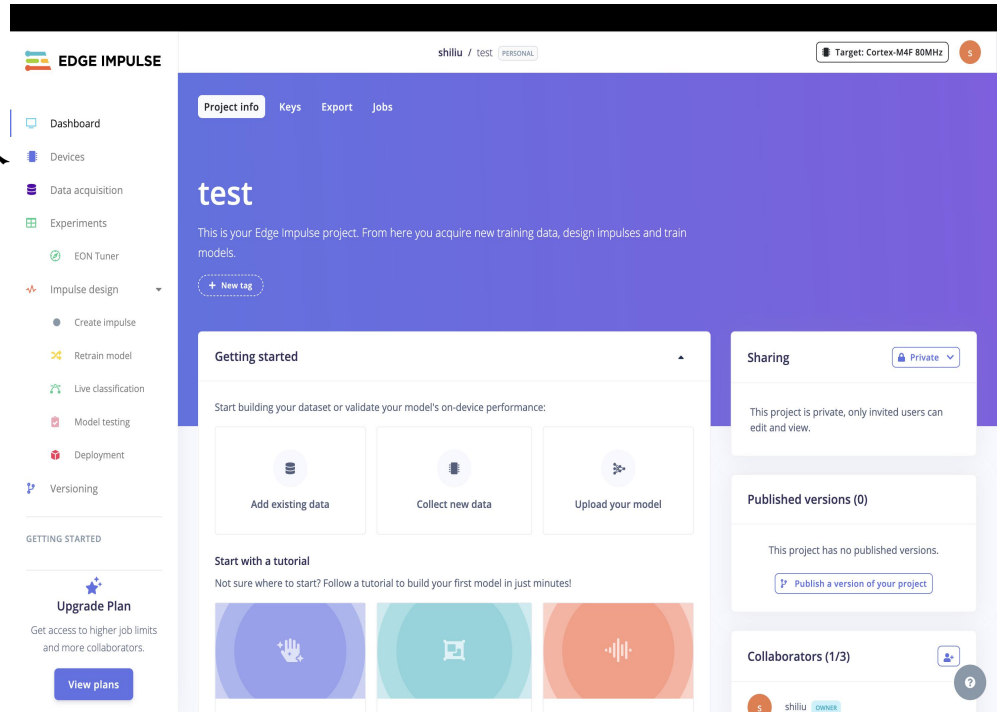
InnoLegend

- 推荐使用Edge Impulse 完成数据标注与模型训练，适合嵌入式平台，简单易用，Edge Impulse：https://edgeimpulse.com/

- 使用OpenMV IDE菜单栏中的数据集编辑器获取数据集

- 操作流程

  注册登录->创建项目->选择开发板->更改标注方法为Bounding boxs（目标检测）->导入数据集->标注数据集



3.2 模型训练与导出

InnoLegend

- 设定Create impulse参数
  - 图像输入需为正方形，使用默认的96*96即可，更高的参数可能导致运行帧率降低
  - 添加处理块（processing block）和学习块（learning block）
  - 保存（save inpulse）
- 处理image设置并生成要素
  - 设定图像深度并保存，生成要素（generate features）
- 设定模型训练参数（Object detection）
  - 训练模型使用默认的MobileNetV2 0.35即可，学习率保持默认的0.001即可，验证集的比例使用默认的20%即可，训练轮数需根据实际情况确定，可以选择200轮，设定完参数之后保存并开始训练
- 模型部署（Deploment）
  - 模型训练好之后，部署选项选择Openmv library，点击build即可导出下载

3.3 OpenMV中模型的使用

InnoLegend

导出模型后获得3个文件，分别是trained.tflite、labels.txt、和示例文件ei_object_detection.py。将这三个文件全部放入sd卡根目录中，可以在IDE中运行ei_object_detection.py检测模型效果

导入必要的库

```
import  ml, math, uos, gc
```

- ml —MicroPython 上的机器学习推理接口
  - ml.Model(path, load_to_fb=False)：加载 TensorFlow Lite 模型，返回模型实例
  - model.predict(inputs, callback=None)：执行推理，输入为图像列表，输出原始张量，并可通过 callback 进行后处理
- uos —文件与存储
  - uos.stat(filename)：获取文件元信息（包括文件大小），用于判断是否将模型映射至 Framebuffer RAM
- gc —内存管理
  - gc.mem_free()：查询可用堆内存大小，与 uos.stat() 搭配，决定 load_to_fb 参数
- math-数学计算

3.3 OpenMV中模型的使用

InnoLegend

模型加载

```
net = ml.Model("trained.tflite",
load_to_fb=uos.stat('trained.tflite')[6] >
(gc.mem_free() - (64*1024)))
```

- load_to_fb=True 时会把模型文件直接映射进 Framebuffer RAM，适用于大模型；否则读到堆中。
- 这里先用 uos.stat() 和 gc.mem_free() 判断剩余内存，确保最少保留 64 KB 才映射。

标签文件读取

```
labels = [line.rstrip('\n') for line in open("labels.txt")]
```

3.3 OpenMV中的模型使用

InnoLegend

后处理

- 在 OpenMV 上运行 FOMO 风格的 .tflite 目标检测模型时，需要进行后处理（Post-Processing）将模型原始输出（通常是多通道的热力图或特征图）转化为「一系列检测框+置信度」

```
threshold_list = [(math.ceil(min_confidence * 255), 255)]

def fomo_post_process(model, inputs, outputs):
```

- 这部分直接照抄示例文件中的后处理函数就行，具体后处理的转换细节不再讲解

- 后处理函数会针对每个通道（即每种标签）都执行一遍，返回 l[i]：第 i 类的所有 (x,y,w,h,score)，即检测框和置信度

3.3 OpenMV中模型的使用

InnoLegend

```
for i, detection_list in enumerate(net.predict([img], callback=fomo_post_process)):
        if i == 0: continue  # background class
        if len(detection_list) == 0: continue  # no detections for this class?

        print("%s" % labels[i])
        for x, y, w, h, score in detection_list:
            center_x = math.floor(x + (w / 2))
            center_y = math.floor(y + (h / 2))
```

- net.predict([img], callback=fomo_post_process)

  - 将 img 包装成单元素列表交给模型推理。
  - callback 参数指定后处理函数，模型原始输出张量交由 fomo_post_process 转成检测框列表。
  - 返回值是一个二维列表，外层下标 i 对应类别通道，内层是该类别的检测框列表。

第四部分

项目实践

Project Practice

4.1项目实践

InnoLegend

- 训练一个数字识别(0 ~ 8)的模型

- 设计一个算法，能够保证小车向着数字移动过程中判断数字的左右顺序，并且保证一定的鲁棒性。其中数字的个数提前做好确定。

STM32H750 电赛智能车 – 原理与实现

欢迎各位同学交流、指正

InnoLegend 技术团队